DIGITAL IDENTITY STRENGTH CHECKER

PROJECT REPORT

Name – R.V. Kashika

Reg No. – 25BOE10047

INDEX

Sr. No.	Section
1	Problem Statement
2	Objectives
3	System Design (Flowchart & Use Case)
4	Working & Explanation
5	Full Source Code
6	Output Screenshots
7	Conclusion
8	References

PROBLEM STATEMENT

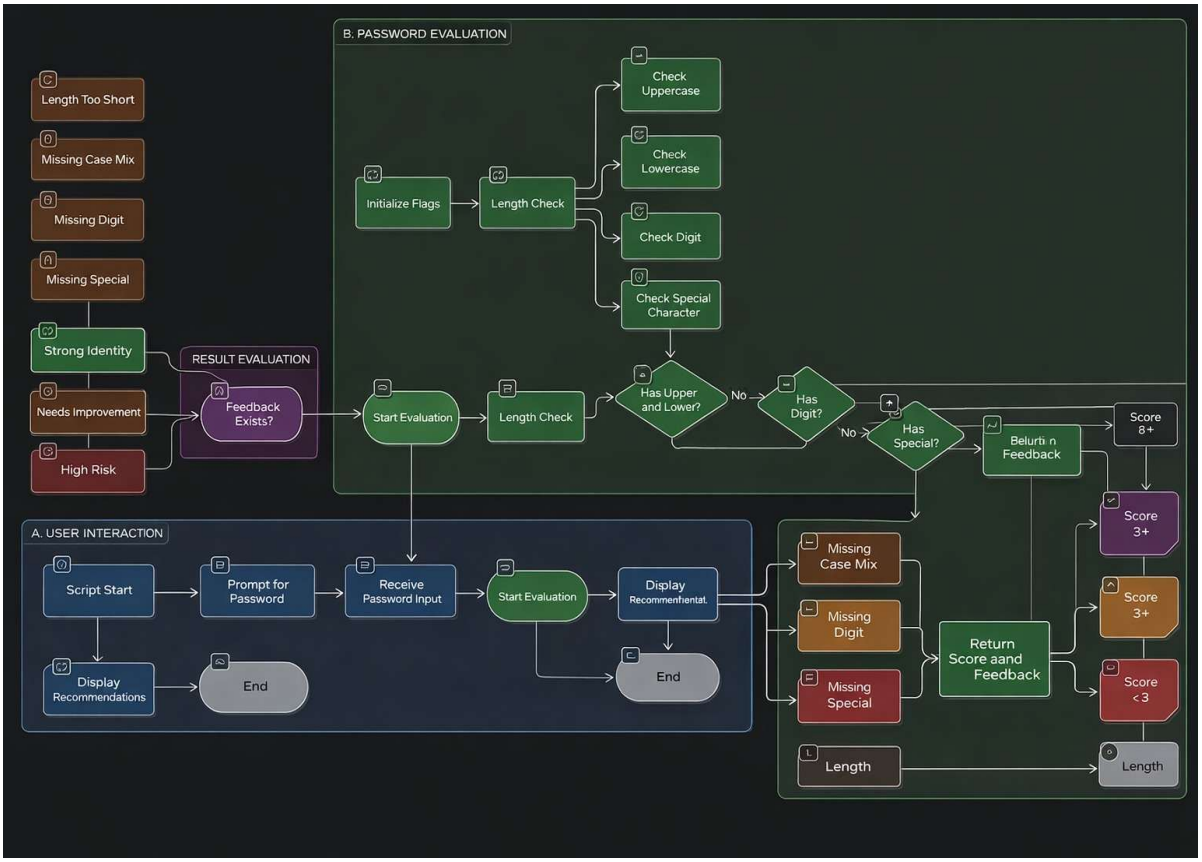
Passwords are used to strengthen the digital security for various websites and apps like GitHub, LinkedIn, email and so on. But these passwords, if made very simple, can easily be identified by hackers through trial-and-error methods, and this can lead to data leaks of even confidential information. To prevent this from happening, this Python program will analyse and check if the password meets the complexity requirements like usage of special characters, case variety, numbers and the overall length.

OBJECTIVES

To design and implement an automated password strength checker that evaluates a user's password against specific security criteria to reduce the risk of cyberattacks.

SYSTEM DESIGN

FLOWCHART



WORKING AND EXPLANATION

The program allows the user to enter a password. Using Boolean logic, it checks if a password is weak or not. If it's weak, then it's flagged, and changes need to be made. Certain criteria for checking include the length of the password, the usage of uppercase and lowercase letters, the usage of numerical and special characters, etc.

If all the criteria are satisfied, then the program returns with a verdict of strong password.

FULL SOURCE CODE

duplicate password checker > ...

```python
# checking the weak passwords for their GitHub or LinkedIn profiles
def passwd_gl(password):
    score_val1 = 0
    comment_dis_val1 = []

    # 1. Check length (Inside function)
    if len(password) >= 12:
        score_val1 += 2
    elif len(password) >= 8:
        score_val1 += 1
    else:
        comment_dis_val1.append("Length should be at least 8 char long")

    upper_case = False
    lower_case = False
    digit_case = False
    spl_case = False
    special_chars = "!@#$%,.?\":{}|<>"

    # 2. Check character types
    for char in password:
        if char.isupper():
            upper_case = True
```

duplicate password checker > ...

```python
def passwd_gl(password):
    for char in password:
        if char.isupper():
        elif char.islower():
            lower_case = True
        elif char.isdigit():
            digit_case = True
        elif char in special_chars:
            spl_case = True

    # 3. Final Scoring (OUTSIDE the for loop, but INSIDE the function)
    if upper_case and lower_case:
        score_val1 += 1
    else:
        comment_dis_val1.append("It should be a mixture of upper and lower case")

    if digit_case:
        score_val1 += 1
    else:
        comment_dis_val1.append("Min one Numerical should be present")

```

```python
    if spl_case:
        score_val1 += 1
    else:
        comment_dis_val1.append("Min one special character should be present")

    return score_val1, comment_dis_val1

def main():
    # Everything inside main must be indented
    print("="*25)
    print(" Digital Identity check ")
    print("="*25)

    user_input = input("Enter a password to test your identity: ")
    strength_score_val1, suggestions = passwd_gl(user_input)

    print("\n--- Security Audit Results ---")

    if strength_score_val1 >= 5:
        print("Status: Strong Identity")
    elif strength_score_val1 >= 3:
        print("Status: Needs Improvement")
```

```python
    else:
        print("Status: High Risk of Cybercrime")

    if suggestions:
        print("\nRecommendations to Increase Complexity:")
        for note in suggestions:
            print(f"- {note}")

# Standard Python entry point
if __name__ == "__main__":
    main()
```

OUTPUT SCREENSHOTS

```
 Digital Identity check
========================
Enter a password to test your identity: jacksaw

--- Security Audit Results ---
Status: High Risk of Cybercrime

Recommendations to Increase Complexity:
- Length should be at least 8 char long
- It should be a mixture of upper and lower case
- Min one Numerical should be present
- Min one special character should be present
PS C:\Users\RVK\OneDrive\Desktop\python proj>
```

```
 ========================
  Digital Identity check
 ========================
 Enter a password to test your identity: Healthy.person201

 --- Security Audit Results ---
 Status: Strong Identity
 PS C:\Users\RVK\OneDrive\Desktop\python proj>
```

```
 ========================
  Digital Identity check
 ========================
 Enter a password to test your identity: aloeVera8

 --- Security Audit Results ---
 Status: Needs Improvement

 Recommendations to Increase Complexity:
 - Min one special character should be present
 PS C:\Users\RVK\OneDrive\Desktop\python proj>
```

CONCLUSION

The project achieves its goal of verifying the strength of the given passwords successfully. Through this, passwords can be more secure and, as a result, will be extremely helpful in preventing data leaks and thereby keep our information private without it getting breached by cybercriminals and scammers.

RESOURCES

Python official documentation – https://docs.python.org

Stack Overflow – General debugging help